

Private LLM - How To Use

A self-contained private LLM + agent on your Mac (Apple M5, 32 GB). An open model runs locally, served over a localhost-only API, driving a tool-calling agent. Nothing leaves the machine by default. This mirrors the GB10 build report, scaled to Apple Silicon.

1. The architecture

```
open model (Qwen3 8B)
-> Ollama OpenAI endpoint @ 127.0.0.1:11434
-> agent loop: goal -> pick tool -> act -> report
```

Facts live in a retrieval layer (rag.py + kb/), not baked into the model. Behavior can be shaped by fine-tuning (finetune/). This is the report's 'behavior in weights, facts in retrieval' split.

2. One-time setup (already done in this folder)

- Ollama installed via Homebrew, running as a localhost service (brew services start ollama).
- Models pulled into ~/.ollama: qwen3:8b, granite3.3:8b, nomic-embed-text.
- Python env managed by uv (Python 3.11). No global installs.

NOTE If the server is ever down: brew services start ollama (check: curl http://127.0.0.1:11434/api/version)

3. Run the agent

All commands run from inside the 'Private LLM' folder.

```
# the report's calibration task: inventory a folder
uv run private_agent.py --task inventory --dir ./sandbox

# a factual question (answered from the knowledge base)
uv run private_agent.py --goal "What port is billing on? Use retrieve."

# let the agent write a file (opt-in, sandboxed)
uv run private_agent.py --allow-write \
  --goal "Read people.csv and write a summary to summary.txt."
```

4. The tools the agent has

- list_dir, read_file, file_info - always on, read-only.
- retrieve - always on, searches the local knowledge base (kb/).
- write_file - opt-in with --allow-write, writes inside the sandbox only.

CAUTION fetch_url - opt-in with --allow-web. THIS LEAVES THE BOX and breaks the sovereignty story. Use only when you intend network access.

5. The knowledge base (RAG)

Facts go in kb/ as plain text/markdown. Re-index after editing:

```
uv run rag.py ingest --dir ./kb # rebuild the index
uv run rag.py query "who is the architect?" # search it directly
```

The agent's retrieve tool searches this same index, so updating a fact is just editing a file in kb/ and re-ingesting - no retraining.

6. Fine-tuning (the next phase)

Tune the model's behavior (cite sources, refuse to guess) while keeping facts in RAG. QLoRA via MLX on Apple Silicon:

```
cd finetune
./run_finetune.sh          # writes LoRA adapters to ./adapters
```

See finetune/README.md to fuse the adapters and serve the tuned model through the same local endpoint.

7. Swapping the model

```
PRIVATE_LLM_MODEL=granite3.3:8b uv run private_agent.py --task inventory
```

Default is qwen3:8b because it drives the tool loop reliably. granite3.3:8b is faithful to the report but flaky through Ollama (see troubleshooting).

8. Troubleshooting / findings

- Agent hallucinates instead of calling tools: this is the report's core finding for small (8B) models. Prefer qwen3:8b; keep guidance in the user turn, not a heavy system prompt (that suppresses tool calls).
- Model invents file sizes/contents: it skipped a tool. Trust only what the tool trace shows; verify anything an 8B asserts unsupervised.
- Connection refused: the Ollama service isn't running - start it (section 2).

9. Sovereignty posture

By default everything is localhost-bound and read-only: the model endpoint and the agent never touch the network. Writes are opt-in and sandboxed; web access is opt-in and clearly flagged as leaving the box. That is governance by construction - the same principle as the report.